

ONEDRAFT

CAD-AI — Aria Powered

Product Configuration & Administrative Control Plane Architecture

Version 1.0

Document: OneDraft Product Configuration & Administrative Control Plane Architecture Specification

System: OneDraft CAD-AI

Intelligence Layer: Aria

Document Class: Architectural & Implementation Stack Specification

Status: Architectural Specification

Predecessor: Developer Platform, SDK & Extension Architecture Specification

Successor: OneDraft End-to-End System Integration Specification

1. PURPOSE

The Product Configuration & Administrative Control Plane establishes the authoritative administrative architecture through which OneDraft platform operators manage controlled system configuration, product behavior, feature availability, jurisdiction configuration, templates, policies and operational settings.

The purpose of this subsystem is to separate controlled administrative configuration from application execution while ensuring that configuration changes are authenticated, authorized, versioned, validated, auditable and safely propagated throughout the platform.

2. ARCHITECTURAL ROLE

The Administrative Control Plane provides the governance layer for configurable OneDraft behavior.

It connects administrative users and authorized automation to controlled configuration resources used by:

Application Services

Aria

Compliance

Validation

Documentation

CAD Standards

Billing

Multi-Tenancy

API Protection

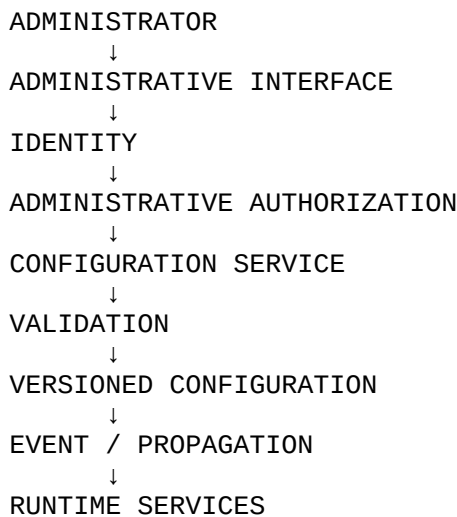
Infrastructure

Developer Platform

The control plane shall not bypass the domain architecture.

3. CONTROL PLANE MODEL

The administrative architecture shall follow:



Administrative configuration shall be treated as controlled system state.

4. ADMINISTRATIVE DOMAINS

The control plane shall manage defined administrative domains.

These may include:

Product Configuration

Feature Flags

Jurisdiction Configuration

Code Sources

Templates

CAD Standards

AI Configuration

Billing Configuration

Organization Policies

System Limits

Integration Configuration

Security Policies

5. CONFIGURATION AUTHORITY

Each configuration value shall have a defined authority.

Configuration shall identify:

Configuration Key

Value

Data Type

Scope

Version

Effective Date

Status

Owner

Created By

Updated By

The system shall prevent ambiguous configuration authority.

6. CONFIGURATION HIERARCHY

Configuration shall support controlled inheritance.

A representative hierarchy is:

PLATFORM
↓
ENVIRONMENT
↓
ORGANIZATION
↓
PROJECT
↓
RESOURCE

More specific configuration may override broader configuration only where the applicable policy explicitly permits inheritance.

7. CONFIGURATION TYPES

Configuration shall support typed values.

Supported categories may include:

Boolean

Integer

Decimal

String

Enumeration

JSON Object

JSON Array

Structured Domain Configuration

Typed configuration shall be validated before activation.

8. CONFIGURATION VALIDATION

Administrative changes shall pass validation before becoming effective.

Validation shall verify:

Data Type

Allowed Values

Required Fields

Dependency Rules

Security Constraints

Compatibility

Scope

Invalid configuration shall not be activated.

9. CONFIGURATION VERSIONING

Configuration changes shall create versioned records.

A configuration version shall identify:

Version

Previous Version

Change

Author

Timestamp

Reason

Status

Effective Time

Configuration history shall remain available for audit and rollback.

10. CONFIGURATION CHANGE LIFECYCLE

Configuration changes shall follow:

DRAFT
↓
VALIDATE
↓
REVIEW
↓
APPROVE
↓
SCHEDULE
↓
ACTIVATE
↓
VERIFY
↓
RETAIN

Administrative policies may omit review for low-risk settings where explicitly authorized.

11. CHANGE CONTROL

High-impact configuration shall require controlled change procedures.

High-impact examples include:

Security Policies

Authorization Rules

Jurisdiction Rules

Billing Rules

AI Provider Configuration

Rate Limits

System Capacity Limits

Production Feature Activation

Changes shall be attributable to an authorized administrative identity.

12. FEATURE FLAGS

Feature flags shall provide controlled product capability activation.

Feature flags may support:

Platform-Wide Activation

Environment Activation

Organization Activation

Project Activation

Percentage Rollout

User-Specific Testing

Feature flags shall be evaluated server-side for security-sensitive behavior.

13. FEATURE FLAG STATES

A feature flag may have states including:

Disabled

Enabled

Scheduled

Deprecated

Retired

State transitions shall be auditable.

14. FEATURE FLAG SAFETY

Feature flags shall not be used to bypass:

Authentication

Authorization

Tenant Isolation

Data Integrity

Compliance Validation

Security Controls

Flags controlling critical security behavior shall require elevated authorization.

15. JURISDICTION CONFIGURATION

The control plane shall manage jurisdiction configuration used by the Compliance & Regulatory Engine.

Jurisdiction records may identify:

Jurisdiction

Authority

Geographic Scope

Applicable Codes

Effective Dates

Code Versions

Source References

Status

Jurisdiction configuration shall remain versioned and provenance-linked.

16. CODE SOURCE CONFIGURATION

Administrative users shall manage approved regulatory sources.

Source records shall include:

Source Identifier

Authority

Publication

Version

Effective Date

Retrieval Date

Integrity Hash

Provenance

Status

The control plane shall not permit unvalidated regulatory sources to become authoritative automatically.

17. CODE MANIFEST CONTROL

Code Manifest configuration shall remain immutable once formally published.

Corrections shall create a new version rather than modifying an existing authoritative manifest.

The administrative control plane shall maintain the relationship between:

Source

Manifest

Rules

Applicability

Effective Period

Validation Results

18. TEMPLATE MANAGEMENT

The control plane shall manage controlled templates.

Templates may include:

Drawing Templates

Sheet Templates

Title Blocks

Annotation Templates

Schedule Templates

Report Templates

Export Templates

Templates shall be versioned and associated with applicable standards.

19. CAD STANDARD CONFIGURATION

Administrative users shall manage approved graphic standards.

Configuration may include:

Layers

Lineweights

Line Types

Text Styles

Dimension Styles

Symbols

Hatching

Sheet Sizes

Plot Configuration

CAD configuration shall integrate with the Standards, Architectural Graphics & CAD Interoperability architecture.

20. DOCUMENTATION CONFIGURATION

Documentation settings may define:

Drawing Naming

Sheet Numbering

View Standards

Annotation Standards

Dimension Standards

Schedule Formats

Document Packages

Configuration shall not override project-specific authoritative requirements without an explicit rule permitting such behavior.

21. ARIA CONFIGURATION

Administrative control shall support controlled Aria configuration.

Configuration may include:

Model Providers

Model Selection Policies

Prompt Versions

Context Policies

Token Limits

Tool Permissions

AI Feature Flags

Evaluation Policies

Aria configuration shall remain subordinate to deterministic command validation and security controls.

22. MODEL PROVIDER CONFIGURATION

AI provider records may include:

Provider

Model

Endpoint

Capability

Availability

Cost Class

Context Limit

Status

Provider credentials shall remain in the Secrets & Environment Management subsystem and shall not be stored as ordinary configuration values.

23. ORGANIZATION CONFIGURATION

Organization administrators may manage authorized organization-level settings.

These may include:

Organization Name

Project Policies

User Policies

Default Templates

Allowed Integrations

Usage Policies

Feature Access

Organization configuration shall remain subordinate to platform-level policy.

24. PROJECT CONFIGURATION

Project-level configuration may include:

Project Standards

Template Selection

Jurisdiction

Drawing Configuration

Validation Configuration

Integration Settings

Project configuration shall not bypass organization or platform security policies.

25. BILLING CONFIGURATION

Administrative controls may manage:

Plans

Entitlements

Usage Limits

Metering Rules

Feature Availability

Subscription States

Billing configuration shall integrate with the Billing, Subscription & Entitlement Architecture.

26. SYSTEM LIMITS

Administrative users may configure controlled system limits.

Examples include:

Maximum Project Size

Maximum Upload Size

Maximum Concurrent Jobs

API Rate Limits

AI Request Limits

Storage Quotas

Worker Concurrency

Limits shall have defined minimum and maximum safe boundaries.

27. RATE-LIMIT CONFIGURATION

Rate limits shall be configurable according to:

Platform

Organization

User

API Key

Endpoint

Integration

Administrative changes shall preserve the API Gateway and Service Protection architecture.

28. SECURITY POLICY CONFIGURATION

Security policies may include:

Session Duration

Authentication Requirements

Credential Expiration

API Key Policies

Password Policies

Administrative Requirements

Access Restrictions

Security configuration shall require appropriate administrative authority.

29. ADMINISTRATIVE ROLES

Administrative permissions shall be separated according to responsibility.

Roles may include:

Platform Administrator

Security Administrator

Billing Administrator

Compliance Administrator

Organization Administrator

Developer Administrator

Support Administrator

No administrative role shall automatically receive unrestricted access unless explicitly defined by platform policy.

30. PRIVILEGED OPERATIONS

Privileged configuration operations shall require elevated authorization.

Examples include:

Security Configuration

Code Manifest Publication

AI Provider Activation

Billing Policy Changes

Platform Feature Activation

Tenant Policy Changes

Administrative Role Changes

Privileged actions shall be fully auditable.

31. ADMINISTRATIVE AUDIT

Every significant administrative change shall generate an audit record.

Audit information shall include:

Actor

Action

Resource

Previous Value

New Value

Timestamp

Request Identifier

Reason

Approval, where applicable.

Audit records shall be immutable.

32. CONFIGURATION DIFF

The control plane shall support configuration comparison.

Administrative users shall be able to identify:

Added Values

Removed Values

Changed Values

Version Differences

Effective Changes

Configuration diffs shall support review and troubleshooting.

33. CONFIGURATION ROLLBACK

Where technically safe, configuration shall support rollback to a prior valid version.

Rollback shall create a new configuration event referencing the restored version rather than rewriting history.

Historical records shall remain intact.

34. SCHEDULED CONFIGURATION

Administrative changes may be scheduled for future activation.

Scheduled changes shall include:

Activation Time

Target Scope

Configuration Version

Approving Identity

Expected Effect

The system shall verify the configuration again at activation where necessary.

35. CONFIGURATION PROPAGATION

Configuration changes shall propagate through controlled mechanisms.

Propagation may use:

Events

Cache Invalidation

Configuration Refresh

Service Restart

Deployment

The propagation mechanism shall be defined by configuration type.

36. CONFIGURATION CONSISTENCY

Services shall identify the configuration version they are using where operationally necessary.

Critical configuration shall not silently diverge between service instances.

Configuration propagation failures shall generate detectable operational events.

37. ENVIRONMENT CONFIGURATION

Development, testing, staging and production shall maintain distinct configuration boundaries.

Production configuration shall not be unintentionally inherited from development environments.

Environment-specific values shall remain under the Configuration, Secrets & Environment Management architecture.

38. ADMINISTRATIVE API

The control plane shall expose controlled administrative APIs.

Administrative API resources may include:

Configurations

Feature Flags

Jurisdictions

Code Sources

Templates

Policies

Providers

Limits

Integrations

Administrative APIs shall enforce the same authorization boundaries as the administrative interface.

39. ADMINISTRATIVE UI

The administrative interface shall provide controlled views for:

Configuration

Version History

Feature Flags

Jurisdictions

Templates

Policies

System Limits

Integrations

Audit Records

Sensitive values shall be appropriately masked.

40. CONFIGURATION CACHE

Frequently accessed configuration may be cached.

Cached configuration shall include a version identifier.

Changes shall invalidate affected cache entries.

Security-sensitive configuration shall use appropriate expiration and invalidation controls.

41. CONFIGURATION INTEGRITY

Configuration records shall support integrity verification.

Critical configuration may include:

Content Hash

Version Hash

Digital Signature, where required.

Integrity verification shall detect unexpected modification.

42. ADMINISTRATIVE WORKFLOW

High-risk administrative changes may require multiple stages.

```
REQUEST
↓
REVIEW
↓
VALIDATE
↓
APPROVE
↓
ACTIVATE
↓
VERIFY
↓
AUDIT
```

The workflow shall support separation of duties where required.

43. ADMINISTRATIVE NOTIFICATIONS

The system may notify authorized administrators about:

Configuration Changes

Failed Activations

Security Policy Changes

Code Updates

Feature Rollouts

Capacity Changes

Integration Changes

Notifications shall not expose secrets or unauthorized tenant information.

44. ADMINISTRATIVE TESTING

Administrative functionality shall be tested for:

Authorization

Configuration Validation

Versioning

Rollback

Propagation

Tenant Isolation

Auditability

Concurrency

Failure Recovery

Security

Administrative testing shall integrate with the Disaster Testing, Security Testing & Operational Readiness architecture.

45. CONFIGURATION DEPENDENCIES

Configuration dependencies shall be explicitly modeled.

For example:

```
FEATURE
  ↓
ENTITLEMENT
  ↓
AUTHORIZATION
  ↓
SERVICE
  ↓
RUNTIME
```

A configuration change shall not activate a capability whose required dependencies are unavailable.

46. CONTROL PLANE OBSERVABILITY

Administrative operations shall produce telemetry.

Metrics may include:

Configuration Changes

Activation Failures

Rollback Events

Feature Flag Evaluations

Administrative API Requests

Authorization Failures

Propagation Failures

Configuration Cache Errors

Observability shall integrate with the system-wide telemetry architecture.

47. ARCHITECTURAL INVARIANTS

The following invariants shall govern the Product Configuration & Administrative Control Plane.

Controlled Authority: Every configuration value shall have a defined authority.

Versioned State: Significant configuration changes shall be versioned.

Auditability: Administrative actions shall remain traceable.

Authorization: Administrative capabilities shall require explicit authorization.

Tenant Isolation: Organization configuration shall never cross tenant boundaries.

Integrity: Critical configuration shall support integrity verification.

Rollback: Safe configuration changes shall support controlled rollback.

Propagation: Configuration changes shall propagate through defined mechanisms.

Validation: Invalid configuration shall not become authoritative.

Separation of Duties: High-risk administrative actions may require independent review or approval.

No Security Bypass: Configuration mechanisms shall not circumvent security, authorization or compliance controls.

Deterministic Governance: Administrative configuration shall remain subordinate to authoritative domain and validation rules.

48. IMPLEMENTATION REQUIREMENTS

The implementation shall provide:

Configuration Service

Administrative API

Administrative Interface

Configuration Schema

Configuration Versioning

Feature Flag System

Jurisdiction Registry

Code Source Registry

Code Manifest Controls

Template Registry

CAD Standards Configuration

Aria Configuration

Provider Registry

Organization Configuration

System Limits

Administrative Roles

Audit Records

Configuration Diff

Rollback

Scheduled Activation

Configuration Propagation

Integrity Verification

Administrative Telemetry

These components shall integrate with Identity, Authorization, Multi-Tenancy, Compliance, CAD Standards, Aria, Billing, API Protection, Observability, Configuration and Infrastructure architectures.

49. CONTROL PLANE GOVERNANCE

The Administrative Control Plane shall remain a controlled subsystem rather than an unrestricted mechanism for modifying application behavior.

Configuration shall be governed according to:

Authority

Scope

Risk

Version

Effective Date

Dependencies

Approval Requirements

Audit Requirements

Critical configuration shall receive stronger controls than ordinary product preferences.

The control plane shall provide centralized governance while preserving the autonomy and integrity of the domain services that consume its configuration.

50. COMPLETION STATUS

The Product Configuration & Administrative Control Plane Architecture establishes the administrative governance layer for OneDraft.

The subsystem defines configuration authority, hierarchy, typed configuration, validation, versioning, change control, feature flags, jurisdiction configuration, regulatory source management, Code Manifest control, templates, CAD standards, documentation configuration, Aria configuration, model provider configuration, organization and project policies, billing configuration, system limits, rate limits, security policies, administrative roles, privileged operations, auditability, configuration diffs, rollback, scheduling, propagation, consistency, environment separation, administrative APIs, administrative interfaces, integrity controls, workflow, notifications, testing, dependencies and operational observability.

The architecture establishes the controlled administrative foundation required to operate OneDraft as a configurable multi-tenant CAD-AI platform while preserving authorization, provenance, security, deterministic domain behavior and system integrity.

The next architectural layer is **50.0 — OneDraft End-to-End System Integration Specification**, which consolidates the preceding architectural stacks and establishes how the complete OneDraft platform operates as one coherent system.

Status: ARCHITECTURALLY DEFINED

Next Document: OneDraft End-to-End System Integration Specification

Overall Architecture: OneDraft End-to-End System Integration